

Program 1:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp1.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    printf("PID = %d\n", getpid());
    return 0;
}

himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp1
PID = 538
```

Program 2:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp2.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    fork();
    printf("Hello\n");
    return 0;
}

himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp2
Hello
Hello
```

Program 3:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp3.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
        printf("I am Child\n");
    else
        printf("I am Parent\n");
    return 0;
}

himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp3
I am Parent
I am Child
```

Program 4:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp4.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
    {
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp4
Parent PID = 546
Child PID = 547
Child PID = 547
Parent PID = 546
```

Program 5:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp5.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child Executing\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Executing\n");
    }
    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp5
Child Executing
Parent Executing
```

Program 6:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp6.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    fork();
    fork();
    printf("PID = %d\n", getpid());
    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp6
PID = 552
PID = 553
PID = 554
PID = 555
```

Program 7:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp7.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child executing ls...\n");
        execl("/bin/ls", "ls", "-l", NULL);
    }
    else
    {
        printf("Parent Waiting...\n");
    }
    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp7
Parent Waiting...
Child executing ls...
himasree@LAPTOP-41COT5AA:~/OSLab$ total 216
drwxr-xr-x 3 himasree himasree 4096 Aug  1 03:05 MYProject
drwxr-xr-x 7 himasree himasree 4096 Aug  1 03:33 MyProject
drwxr-xr-x 2 himasree himasree 4096 Jul 31 09:22 exec
-rwxr-xr-x 1 himasree himasree 16008 Jul 30 09:58 exp1
-rw-r--r-- 1 himasree himasree  99 Jul 29 03:04 exp1.c
-rwxr-xr-x 1 himasree himasree 16000 Jul 30 09:59 exp2
-rw-r--r-- 1 himasree himasree  94 Jul 29 03:06 exp2.c
-rwxr-xr-x 1 himasree himasree 16000 Jul 30 09:59 exp3
-rw-r--r-- 1 himasree himasree  163 Jul 29 03:07 exp3.c
```

Program 8:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp8.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    int i;
    for(i=0;i<3;i++)
    fork();
    printf("PID = %d\n",getpid());
    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp8
PID = 562
PID = 563
PID = 566
PID = 564
PID = 568
PID = 565
PID = 567
PID = 569
```

Program 9:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat exp9.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    char cmd[50];
    printf("Enter Linux Command: ");
    scanf("%s", cmd);
    pid_t pid = fork();
    if(pid < 0)
    {
        printf("Fork Failed\n");
        return 1;
    }
    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        execlp(cmd, cmd, NULL);
        perror("Command Execution Failed");
        exit(1);
    }
    else
    {
        printf("\nParent Process\n");
```

```
printf("Child PID : %d\n", getpid());
printf("Parent PID: %d\n", getppid());
execlp(cmd, cmd, NULL);
perror("Command Execution Failed");
exit(1);
}
else
{
printf("\nParent Process\n");
printf("Parent PID : %d\n", getpid());
printf("Child PID : %d\n", pid);
wait(NULL);
printf("\nChild Process Completed\n");
}
return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ ./exp9
Enter Linux Command: ls

Parent Process
Parent PID : 571
Child PID : 572

Child Process
Child PID : 572
Parent PID: 571
MyProject exec  exp1.c  exp2.c  exp3.c  exp4.c  exp5.c  exp6.c  exp7.c  exp8.c  exp9.c  program.c
MyProject exp1  exp2   exp3   exp4   exp5   exp6   exp7   exp8   exp9   program
Child Process Completed
```